

PyLocalSend PPT 完整发言稿

配套课件： `doc/pylocalsend-presentation.pptx`

参考提纲： `doc/pylocalsend-speech.md`

建议时长：18~22 分钟（含 2~3 分钟演示视频）

使用说明：正文可以直接照读；方括号中的内容是翻页、停顿或操作提示，不需要念出。

第 1 页：封面

各位老师、同学，大家好。

我们今天汇报的项目是 **PyLocalSend**，它是一个使用 Python 开发的局域网文件传输工具。

这个项目主要解决两类需求：第一，在同一个局域网里，把一台电脑上的文件或文件夹方便地共享给其他设备；第二，在需要收作业、收实验数据时，允许指定的接收端把文件反向上传到发送端。

项目底层使用 FastAPI 提供 HTTP 服务，使用 NiceGUI 构建网页界面，并通过分片和流式读写降低大文件传输时的内存压力。

接下来，我们将从项目背景、产品功能、系统架构、具体实现和演示五个部分进行介绍。

[停顿 1 秒，翻到下一页]

第 2 页：01 / PROBLEM

首先来看我们为什么要做这个项目。

文件传输看起来是一个已经被解决的问题。U 盘、网盘和聊天软件都能传文件。但是在真实使用中，问题往往不在于“能不能传”，而在于传输过程的成本、步骤、速度和可控性。

特别是在教室、机房和实验室这种局域网环境中，我们希望文件传输足够直接，不依赖额外硬件，不需要先上传到公网，也不希望每个接收端都安装一套复杂的软件。

[翻页]

第 3 页：传统方式的阻力

我们先看几种常见方式。

第一种是 U 盘。它的优点是直观，但是依赖实体硬件，需要反复插拔。多人依次复制时效率比较低，也容易漏掉文件或子目录。

第二种是网盘。网盘适合跨网络分享，但在同一个局域网里，文件仍然要先上传到公网服务器，再从公网下载回来。这个过程增加了等待时间，也会带来隐私、账号和网络速度方面的问题。

第三种是聊天软件。它适合发送少量小文件，但经常存在文件大小、过期时间或目录结构方面的限制。发送一个完整工程目录时，通常还要先压缩，再上传，再下载和解压。

因此，PyLocalSend 的目标不是替代所有文件传输工具，而是针对可信局域网中的临时共享场景，尽量减少中转和操作步骤。

[翻页]

第 4 页：传统方案，先复制到内存

这一页展示的是一种比较朴素的文件传输实现。

发送端先读取文件内容，再把读取到的数据通过网络发送给接收端。如果程序一次性读取整个文件，那么文件越大，占用的内存就越多；同时传输开始前，接收端还需要等待发送端完成前面的读取或准备工作。

对于几 MB 的小文件，这种问题可能不明显。但如果传输的是几个 GB 的视频、镜像或数据集，一次性加载就不合适了。

需要说明的是，并不是所有传统工具都会这样实现。这里对比的是“整文件读取和缓存”这种简单方案，目的是说明我们为什么选择流式处理。

[沿图示方向指示：磁盘、内存、网络、接收端；然后翻页]

第 5 页：PyLocalSend 方案，边读边发

PyLocalSend 采用分片流式传输。

发送端收到下载请求以后，不会先把整个文件读入内存，而是按照配置的分片大小读取一部分，发送一部分。接收端也可以边接收边写入本地文件。

这样做有两个直接好处。

第一，内存占用主要和分片大小有关，而不是和整个文件大小一起增长。第二，发送端不需要先生成一份完整的临时副本，传输可以更快开始。

这里准确的说法是“分片流式读取，避免整文件缓存”。它并不是操作系统层面的零拷贝，因为数据分片仍然会经过应用程序内存。

[停顿，翻页]

第 6 页：项目定位

基于前面的分析，我们把项目定位概括为一句话：**发送端负责共享，接收端按授权下载。**

发送端只需要启动服务，并登记本机已有的文件或文件夹路径。这里的“登记”只保存路径和元数据，不会把文件复制到项目目录。

接收端既可以使用命令行，也可以直接打开浏览器。浏览器方式不要求安装 Python，因此比较适合课堂临时分享。

系统不依赖公网网盘，也没有设计复杂的用户账号体系。发送端可以创建多个接收端，为每个接收端生成独立 token 链接，并控制其下载范围、上传权限和启用状态。

这就是 PyLocalSend 的基本边界：它面向可信局域网，强调简单、直接和可管理。

[翻页]

第 7 页：02 / PRODUCT

第二部分介绍产品功能和实际使用方式。

PyLocalSend 提供两个入口：一个是 CLI 命令行，另一个是 WebUI 网页界面。

CLI 更适合明确指定下载目录、批量下载、并行下载和断点续传；WebUI 更适合发送端管理、课堂展示，以及让接收端在零安装的情况下直接使用浏览器完成下载和上传。

这两个入口不是两套独立系统，它们共用同一套核心服务、数据库和权限逻辑。

[翻页]

第 8 页：发送端 WebUI

发送端可以通过 `pylocalsend gui` 启动图形管理界面。

界面主要分为四类功能。

第一类是文件传输。用户输入本机文件或文件夹的绝对路径，将它登记到系统中。登记完成以后，还要勾选需要对接收端开放的文件或子目录。

第二类是接收端管理。发送端可以创建接收端，为它生成带 token 的专属链接，也可以复制链接给另一台电脑。每个接收端都可以单独设置是否允许浏览器上传，还可以随时禁用或重新启用。

第三类是传输记录。发送端可以查看谁在什么时间下载了哪个项目，以及传输了多少数据。

第四类是系统设置，例如服务端口、传输分片大小和浏览器上传文件的存放目录。

因此，这个 WebUI 不只是一个“选择文件并发送”的页面，它同时承担共享范围和接收端权限的管理。

[翻页]

第 9 页：CLI 使用

命令行适合更精确的操作。

例如，`pylocalsend gui` 用于启动管理界面；`pylocalsend upload` 用于在发送端登记共享路径；`pylocalsend ls-remote` 用于查看远端当前允许下载的项目；`pylocalsend download` 用于把指定文件或文件夹下载到目标目录。

和浏览器相比，CLI 的一个明显优势是可以明确指定保存位置。它还支持多个文件并行下载，并通过 HTTP Range 实现断点续传。

也就是说，如果只是临时接收一个文件，打开网页最方便；如果需要稳定地下载很多文件，或者下载体积很大的内容，CLI 会更合适。

[如果时间紧，只介绍 `upload`、`ls-remote` 和 `download`；然后翻页]

第 10 页：接收端 WebUI

接收端拿到发送端生成的完整专属链接以后，可以直接在浏览器中打开，例如 `http://192.168.1.100:8765/r/<token>`，不需要安装 PyLocalSend。这里的 `<token>` 会由系统替换成一段随机生成的实际字符串，不需要用户手动填写。

对于单个文件，浏览器会直接向发送端发起下载请求，发送端以流的方式返回文件内容。

对于文件夹，浏览器本身不方便递归保存完整目录，所以发送端会根据当前开放范围，实时生成 ZIP 数据流。接收端最终会得到一个以文件夹名称命名的 ZIP 文件。

这里的重点是，发送端不会先在磁盘上生成一个完整的临时 ZIP 包，再把它发送出去，而是在打包的同时持续输出数据。

接收端页面只显示已经获得授权的内容。页面还会定期检查 catalog 版本；当发送端修改开放范围以后，接收端列表可以自动更新。

[翻页]

第 11 页：接收端浏览器上传

除了从发送端下载，我们还实现了浏览器反向上传。

这个功能适合老师收作业、实验室收数据，或者临时把另一台设备上的文件集中到发送端。

为了避免任何拿到链接的人都能上传，上传权限默认关闭。发送端必须在接收端管理页面中，对某个接收端单独点击“允许上传”。权限开启以后，对方页面才会出现“上传文件”标签。

接收端可以通过 NiceGUI 的上传组件选择一个或多个文件。文件最终保存在发送端设置的上传根目录下，并按照“接收端名称”建立子目录。默认根目录是 `~/.pylocalsend/browser_uploads`。

如果出现同名文件，系统会在新文件名中加入时间戳，避免直接覆盖原文件。

还有一个重要设计：上传成功并不等于自动共享。上传文件只会写入发送端磁盘，不会自动加入共享列表。发送端如果希望其他人下载，仍然需要手动登记路径并设置开放范围。

[强调“上传不等于共享”，翻页]

第 12 页：下载开放与接收端控制

这一页把权限控制集中展示出来。

首先是开放范围。发送端通过勾选来设置 grants，也就是授权路径。对于一个文件夹，可以开放整个文件夹，也可以只开放其中某几个子目录或文件。接收端查看列表、获取目录树、下载单文件和下载 ZIP 时，都会受到这套授权规则限制。

第二是接收端状态。发送端可以一键禁用某个接收端。禁用以后，它不能发起新的下载或上传；如果它正在下载，服务端也会触发取消。重新启用后，原来的链接可以继续使用。

第三是上传权限。它和下载权限分开控制，必须针对接收端单独开启。

第四是自动同步。当授权范围发生变化时，catalog 版本也会变化，接收端网页通过轮询发现变化并刷新内容。

所以，PyLocalSend 的权限模型虽然不复杂，但覆盖了“谁可以访问、能看到什么、能否上传、是否立即停用”几个核心问题。

[翻页]

第 13 页：03 / ARCHITECTURE

第三部分介绍系统架构。

如果用一句话概括 PyLocalSend 的工作方式，就是：**路径登记、按需读取、流式返回**。

文件注册阶段只保存路径和元数据；真正收到下载请求以后，服务端才打开源文件并按分片读取；读取到的数据直接进入 HTTP 响应流。

这种设计避免了不必要的文件副本，也让 CLI 和浏览器能够使用同一套下载接口。

[翻页]

第 14 页：整体架构

从整体上看，系统分为发送端和接收端。

左侧是发送端。核心的 SenderService 负责业务逻辑，FastAPI 提供 HTTP API，NiceGUI 提供管理页面和接收端页面，SQLite 保存共享元数据、授权范围、接收端信息和下载记录。本地文件仍然保存在原来的磁盘位置。

右侧是接收端。接收端可以使用 CLI，也可以使用普通浏览器。CLI 会把数据写入用户指定的目录；浏览器则使用浏览器自己的下载机制。

中间通过 HTTP 通信。鉴权可以使用 PIN，也可以使用接收端 token。单文件下载支持 Range 请求，文件夹可以使用 ZIP 流，浏览器上传则是一个默认关闭的可选能力。

这个架构没有额外的中心服务器，发送端本身就是局域网内的服务节点。

[翻页]

第 15 页：文件共享流程

一条文件从登记到下载，大致经过六个步骤。

第一，发送端启动服务。

第二，用户登记一个本机文件或文件夹路径。

第三，系统提取名称、路径、类型和大小等元数据，并写入 SQLite；文件内容本身不会写入数据库。

第四，接收端通过 API 获取自己有权看到的项目列表。

第五，接收端选择某一项并发起下载请求。

第六，发送端完成鉴权和授权检查，打开源文件，按分片读取并通过 HTTP 返回。

整个流程的三个关键词是：注册时不复制，下载前不生成完整缓存，传输时按分片处理。

[按图从左到右指示，翻页]

第 16 页：文件夹下载策略

文件夹的处理方式需要区分 CLI 和浏览器。

CLI 会先获取目录树，在接收端创建对应的目录结构，然后递归下载其中的文件。这种方式可以保留目录结构，并结合并行下载和 Range 续传，适合工程目录或大量文件。

浏览器无法像本地程序一样自由创建任意目录结构，所以 WebUI 使用 ZIP 流。发送端遍历当前获准下载的文件，把它们逐个写入 ZIP 输出流，并把生成的数据持续发送给浏览器。

这里使用的是流式打包，因此发送端不会先生成一个完整的临时压缩包。当前实现采用 ZIP 存储模式，重点是边打包边传输和保持较低的额外磁盘占用。

两种方式服务于不同场景：CLI 更灵活，WebUI 更简单。

[翻页]

第 17 页：访问控制与记录

系统的访问控制和记录主要由六个部分组成。

第一，PIN 用于基础身份验证。

第二，TOKEN 用于标识独立接收端。每个接收端有自己的链接，也可以被单独禁用。

第三，GRANT，也就是下载授权，决定具体哪些路径可见、可下载。

第四，UPLOAD，决定某个接收端是否允许通过浏览器反向上传。

第五，LOG，记录下载的文件、接收端、开始时间、状态和已发送字节数。

第六，VERSION，也就是 catalog 版本。它根据共享项目和授权内容生成，用于让接收端 WebUI 感知列表变化。

这些机制组合在一起，让文件共享不只是“把端口开放出来”，而是具备基本的身份、授权、停用和审计能力。

[翻页]

第 18 页：04 / IMPLEMENTATION

第四部分简单介绍代码实现。

项目在结构上尽量让入口层保持轻量。CLI 负责解析命令和展示结果，GUI 负责页面交互，而文件处理、接收端逻辑、发送端服务、鉴权、数据库和配置等核心能力放在 `core` 层。

这样做的好处是，CLI 和 WebUI 可以复用同一套规则。比如接收端禁用、下载授权和日志记录，不会在两个入口中分别实现两遍。

[翻页]

第 19 页：技术栈

项目使用的技术栈以 Python 生态为主。

FastAPI 负责 HTTP API、依赖注入和流式响应；NiceGUI 用于构建发送端和接收端的网页界面；httpx 用于 CLI 的异步 HTTP 请求；Rich 用于命令行表格和下载进度展示；SQLite 用于本地持久化；cryptography 用于 CLI 下载路径中可选的 AES-CTR 加密流。

文件读写部分使用异步和分片处理，目录下载则根据客户端类型选择递归下载或 ZIP 流。

整体没有引入 Redis、消息队列或独立数据库服务，部署时只需要 Python 环境和项目依赖，符合课程项目轻量、可运行、便于理解的目标。

[翻页]

第 20 页：项目结构

接下来按目录看一下代码职责。

`cli` 目录负责命令行入口。

`core/file_handler` 负责文件元数据、目录树、分片读写以及下载授权路径的判断。

`core/receiver` 负责接收端客户端逻辑，`core/transfer` 负责发送端服务和下载接口，`core/utils` 负责配置、数据库、加密、日志、网络地址和端口等基础能力。

`gui/sender` 是发送端管理界面，`gui/receiver` 是接收端浏览器页面，`gui/facade.py` 在 GUI 和核心服务之间提供统一调用接口。

`tests` 目录覆盖 CLI、鉴权、单文件下载、文件夹 ZIP 下载、禁用接收端、浏览器上传和自定义上传目录等关键行为。

如果需要快速阅读核心代码，可以先看 `core/transfer/sender.py`，再看发送端 GUI 的 `gui/sender/app.py`。

[翻页]

第 21 页：数据持久化

PyLocalSend 的运行数据默认保存在用户目录下的 `~/.pylocalsend`。

`config.json` 保存服务端口、分片大小、是否启用加密以及上传目录等配置。

`session.json` 保存 CLI 最近使用的连接信息，减少重复输入。

`pylocalsend.db` 是 SQLite 数据库。`files` 表保存登记的共享路径；`download_grants` 保存每个共享项目的开放范围；`receivers` 保存接收端名称、PIN、token、状态和上传权限；`download_logs` 保存下载记录。

数据库中保存的是元数据和控制信息，不保存共享文件内容。

浏览器上传的文件也不会自动写入 `files` 表。它只会被保存到配置目录，之后是否共享由发送端再次决定。

[翻页]

第 22 页：05 / DEMO

前面介绍了产品和实现，接下来进入演示部分。

我们将展示一个完整流程：启动发送端 GUI，登记并开放文件，创建接收端链接，在浏览器中下载文件，然后为接收端开启上传权限并完成一次反向上传。

演示中请重点关注两点：接收端只能看到已开放的内容；接收端上传成功后，文件不会自动变成共享项目。

[切换到视频页]

第 23 页：演示视频

[开始播放视频]

现在启动 `pylocalsend gui`，打开发送端管理页面。

首先进入文件传输页面，输入本机路径并完成登记。可以看到，这一步只是登记路径，并没有把源文件复制到项目目录。

接下来勾选需要开放的文件或子目录。没有勾选的内容不会出现在接收端列表中。

然后进入接收端管理页面，创建一个新的接收端。系统生成带 token 的专属链接，我们把它复制到另一个浏览器窗口中。

接收端打开链接以后，可以看到刚才获得授权的项目。选择单文件时，浏览器直接下载；选择文件夹时，发送端返回 ZIP 流。

现在回到发送端，为这个接收端开启上传权限。接收端页面刷新后，会出现“上传文件”标签。

选择文件并上传后，文件会保存到发送端配置的上传目录，并放在当前接收端名称对应的子目录中。

最后回到共享列表。可以看到，刚刚收到的文件不会自动出现在这里。如果还需要继续分享，发送端必须手动登记该路径并设置开放范围。

[视频结束]

以上就是 PyLocalSend 的主要操作流程。

[翻页]

第 24 页：当前限制

最后，我们也说明一下项目目前的限制。

第一，浏览器下载的保存位置受浏览器安全模型控制。WebUI 不能像 CLI 一样直接指定任意绝对路径。

第二，项目默认面向可信局域网。它提供了 PIN、token 和授权控制，但没有把自己设计成可以直接暴露到公网的文件托管服务。如果需要公网使用，还要增加 HTTPS、反向代理、访问频率限制和更完整的安全审计。

第三，浏览器上传目前不支持断点续传，并且上传组件在处理超大文件时会受到浏览器和服务端内存机制的影响。因此，大文件上传更适合后续扩展为分片上传。

第四，上传文件不会自动共享。这是有意的安全设计，但也意味着发送端需要多做一次登记操作。

因此，目前的使用建议是：CLI 适合批量、大文件和需要指定目录的下载；WebUI 适合零安装接收、临时分享、反向上传和课堂演示。

[翻页]

第 25 页：总结

最后总结一下我们的项目。

PyLocalSend 面向可信局域网中的文件共享场景。发送端登记已有路径，接收端通过 CLI 或浏览器按授权下载；传输过程采用分片流式读写，避免整文件缓存和不必要的临时副本。

它的核心特点可以概括为六点。

第一，低成本，不需要额外硬件或独立服务器。

第二，局域网直连，不依赖公网网盘中转。

第三，流式传输，适合处理大文件。

第四，可管理，可以创建、禁用接收端并查看下载记录。

第五，细粒度开放，可以控制到文件夹中的具体子路径。

第六，支持按接收端授权的浏览器反向上传。

我们希望这个项目能够在课堂、机房和实验室中，真正减少几步重复操作，同时也成为一个比较完整的 Python 网络编程实践。

[翻页]

第 26 页：分工说明

本项目由李子健和杨秉熹共同完成。

李子健主要负责 GUI 和 CLI 的设计与实现、部分核心业务逻辑，以及演示视频的剪辑。

杨秉熹主要负责部分核心业务逻辑、演示视频素材和 PPT 制作。

在核心功能方面，我们共同完成了需求讨论、接口联调、问题排查和最终测试；在展示材料方面，则根据各自负责的模块进行了分工。

[两位成员可在此点头示意，翻页]

第 27 页：Q&A

我们的汇报到这里结束，谢谢各位老师和同学。

下面进入提问环节，欢迎大家提出问题。

答辩备用回答

1. PyLocalSend 和网盘、聊天软件相比，核心区别是什么？

PyLocalSend 不提供公网存储，而是让发送端直接在局域网内提供服务。文件不需要先上传到第三方服务器，发送端也可以控制每个接收端的链接、状态、下载范围和上传权限。它更适合临时、近距离、可信网络中的文件传输。

2. 为什么不让上传文件自动进入共享列表？

上传和共享被设计成两条独立链路。接收端把文件交给发送端，并不代表这个文件应该立即对其他接收端公开。由发送端再次登记和授权，可以避免误上传文件自动暴露，也让老师在收作业后先检查内容。

3. 所谓流式传输是不是零拷贝？

不是。当前实现会把文件按分片读入应用程序内存，再写入 HTTP 响应，因此准确说法是“分片流式传输”和“避免整文件缓存”。操作系统意义上的零拷贝通常需要 `sendfile` 等机制，当前项目没有把这一点作为实现目标。

4. 大文件传输中断以后怎么办？

CLI 下载支持 HTTP Range。客户端可以根据本地已经写入的字节数，从指定偏移继续请求剩余内容。浏览器普通下载是否能够恢复则取决于浏览器行为，目前不作为项目保证的能力。

5. 文件夹为什么要实时打包成 ZIP？

浏览器不能像本地 CLI 一样自由创建完整目录树。把文件夹转换成 ZIP 流，可以让浏览器通过一次下载保留目录结构，同时避免发送端提前生成完整临时压缩包。CLI 不受这个限制，所以采用递归下载。

6. ZIP 流会不会消耗很多 CPU？

当前实现以流式封装和低额外磁盘占用为主要目标，使用 ZIP 存储模式，不进行高强度压缩。因此 CPU 压力相对可控，但传输体积也不会因为压缩明显减少。

7. token 和 PIN 分别解决什么问题？

PIN 提供基础验证；token 用于标识具体接收端。通过 token，系统可以知道是谁在访问，并对这个接收端单独设置禁用状态和上传权限。下载授权目前主要绑定共享项目和开放路径，所有有效接收端看到的是发送端当前开放的 catalog。

8. 禁用接收端为什么能中断正在进行的下载？

服务端会记录当前活动下载与接收端的对应关系。禁用接收端时，系统为相关下载设置取消信号；流式生成器在发送每个分片前检查该信号，发现被取消后停止继续输出。

9. 如果源文件在传输过程中被修改或删除，会发生什么？

系统按当前路径读取源文件。如果文件被删除、权限变化或读取失败，传输会中断并记录失败状态。如果文件内容在传输中被修改，当前版本没有做文件快照，因此接收结果可能不一致。后续可以增加修改时间检查、校验和或快照机制。

10. 为什么选择 SQLite？

项目是单机发送端工具，数据量主要是共享元数据、接收端信息和下载日志。SQLite 不需要额外部署服务，事务和查询能力足够，也方便把运行数据统一保存在用户目录中。

11. 可以直接部署到公网吗？

不建议直接暴露。项目当前面向可信局域网。公网部署至少需要 HTTPS、反向代理、防火墙、强身份验证、上传大小限制、访问频率限制和更严格的路径与内容安全检查。

12. 后续最值得增加什么功能？

可以从三个方向扩展：第一，浏览器分片上传和断点续传；第二，文件哈希校验和传输完成验证；第三，按接收端配置独立下载授权，而不是让所有有效接收端共享同一份开放 catalog。

双人主讲交接词

第 6 页结束，杨秉熹交给李子健

以上是项目背景和定位。接下来请李子健介绍 PyLocalSend 的具体功能和使用方式。

第 12 页结束，李子健交给杨秉熹

前面介绍的是用户能够看到的功能。下面由杨秉熹从系统内部说明这些功能是怎样组织起来的。

第 17 页结束，杨秉熹交给李子健

这就是项目的整体架构和控制逻辑。接下来请李子健简单介绍代码结构与具体技术实现。

第 23 页演示结束，李子健交给杨秉熹

以上是完整演示。最后由杨秉熹介绍项目目前的限制并进行总结。

时间不足时的压缩方案

如果现场只允许 15 分钟，可按下面方式压缩：

1. 第 3 页只保留 U 盘、网盘、聊天软件各一句。
2. 第 8~12 页重点讲第 10、11、12 页，其余页面控制在 30 秒以内。
3. 第 19~21 页合计控制在 1 分钟，只讲 FastAPI、SQLite 和核心目录。
4. 第 23 页视频控制在 2 分钟以内。
5. 第 26 页分工说明控制在 20 秒。

压缩后仍应保留三处重点：流式传输不是整文件缓存；上传不等于自动共享；禁用接收端会同时阻止下载和上传。